

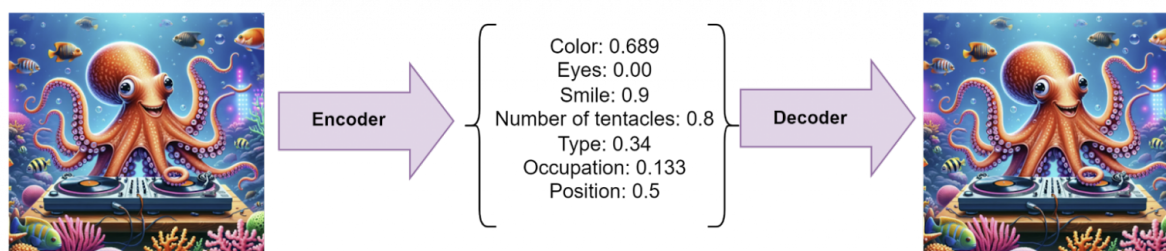
Лабораторная работа: Генерация данных с использованием вариационных автоэнкодеров (VAE)

Автоэнкодеры являются базовой техникой машинного обучения и искусственного интеллекта, на основе которой строятся более сложные модели, например, в диффузионных моделях, таких как Stable Diffusion.

Автоэнкодеры автоматически учатся кодировать данные в более компактное или низкоразмерное представление и затем декодировать его обратно в исходную форму. Этот термин можно разбить следующим образом:

- **Авто:** означает что процесс является автоматическим. Модель учится сама, без необходимости в явных метках или человеческом вмешательстве для извлечения признаков. Во время обучения она автоматически находит, как лучше всего представить данные в низкоразмерной форме.
- **Энкодер:** первая часть модели, энкодер, сжимает или кодирует входные данные в меньшую, сжатую форму. Этот этап включает уменьшение размерности входных данных, фактически обучаясь более компактной версии данных.
- **Декодер:** вторая часть модели, декодер, пытается восстановить исходные входные данные из закодированного представления. Цель состоит в том, чтобы обеспечить максимальное сходство выходных данных с оригинальными входными, показывая, что кодирование сохраняет основные признаки.

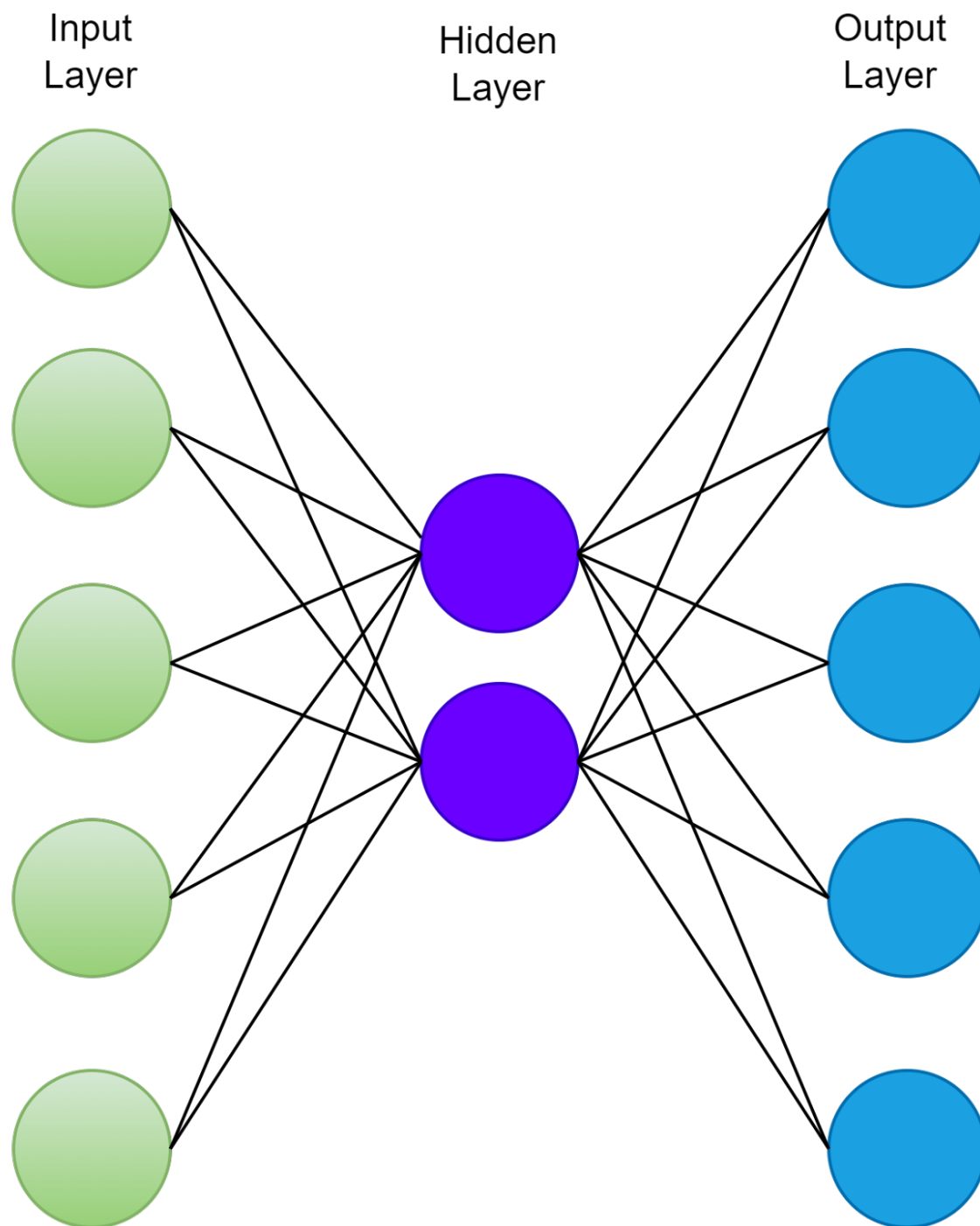
Итак, прежде всего, автоэнкодер — это тип нейронной сети, используемый для обучения без учителя. Но не любой, а такой, который может кодировать и декодировать данные. В машинном обучении он используется для уменьшения размерности или сжатия данных, а также для удаления шума с изображений. Он способен понять самые важные особенности данных (так называемые латентные, или скрытые признаки) и запоминает их вместо всех данных, чтобы затем восстановить что-то близкое к оригиналу из приблизительного описания. На изображениях, например, он может запомнить очертания объектов или относительное положение объектов друг к другу. Это позволяет добиться интересного сжатия с потерями. Он работает примерно следующим образом:



Тут надо указать что то это лишь пример для понимания, и что в реальности распознает сеть будет отличаться. А так же что ни один осьминог при данных экспериментах не пострадал.

Данный подход важен для диффузионных моделей, потому что позволяет им работать в пространстве более низкой размерности (так называемом латентном пространстве), что гораздо быстрее, чем работа напрямую с изображениями высокого разрешения. То есть, вместо того чтобы выполнять процесс удаления

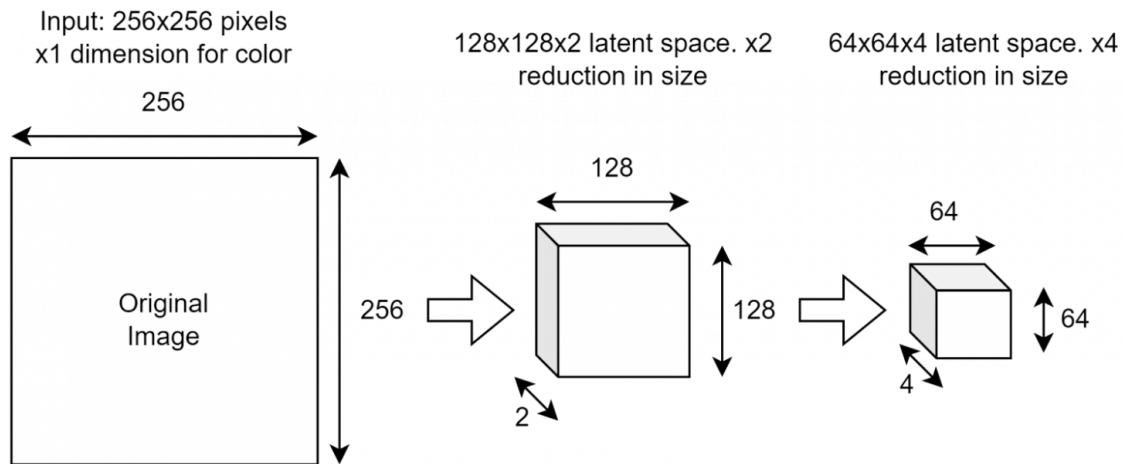
шума непосредственно на пикселях, изображение сначала сжимается в латентное пространство с помощью автоэнкодера, и диффузионный процесс происходит в этом пространстве меньшей размерности. После диффузии декодер восстанавливает изображение высокого разрешения из латентного представления. Для работы диффузионной модели нужно знать только основные признаки оригинального изображения, а не все мельчайшие детали. Если мы искусственно ограничим количество узлов в сети автоэнкодера и обучим её восстанавливать данные из оригинала, это заставит сеть научиться сжато представлять исходные данные просто потому, что у неё не будет достаточно узлов, чтобы запомнить все признаки данных. Ей придётся отбросить большинство несущественных признаков. Это, конечно, возможно только в том случае, если в данных существует какая-то структура (например, корреляции между входными признаками), потому что тогда эту структуру можно выучить и использовать, когда данные проходят через узкое место сети.



Создание узкого места в нейронной сети

Эту сеть можно обучить путем минимизации *ошибки реконструкции*, которая измеряет различия между нашими исходными данными и восстановленным изображением после декомпрессии.

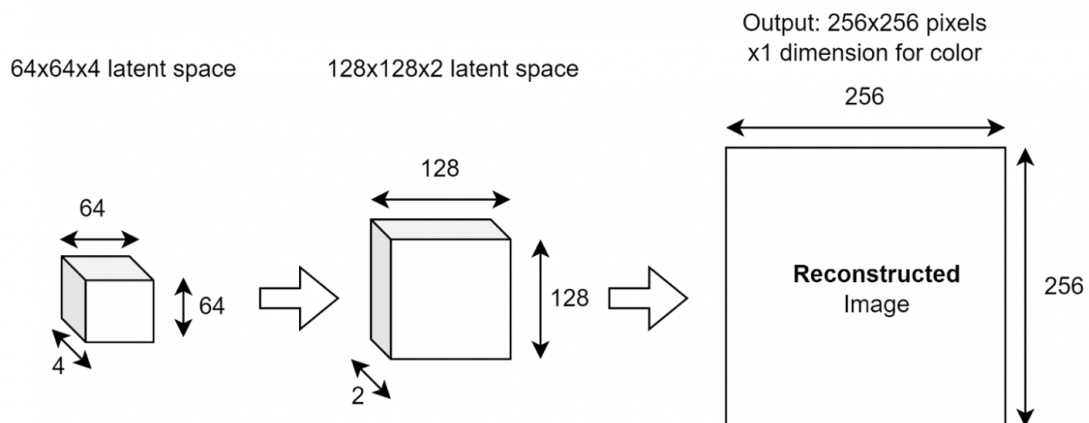
Вот пример того, как работает этап сжатия:



Кодирование

На каждом этапе сжатия мы уменьшаем размеры изображения вдвое, но удваиваем количество каналов, которые сеть может использовать для хранения латентных признаков.

Декодирование работает в обратном порядке:



Декодирование

Обратите внимание, что в конце мы получаем восстановленное изображение, а не оригинал. Они будут похожи, но не идентичны. Да, цель процесса обучения — сделать их как можно более похожими (то есть минимизировать *ошибку реконструкции*), но некоторые детали будут утрачены.

Идеальная модель автоэнкодера находит баланс между:

1. Достаточной чувствительностью к входным данным для их точного восстановления.
2. Достаточной нечувствительностью, чтобы избежать простого запоминания или переобучения на тренировочных данных.

В большинстве случаев это достигается путем задания функции потерь с двумя компонентами: один член, который поощряет модель быть чувствительной к входным данным (например, ошибка реконструкции), и другой член, который препятствует запоминанию или переобучению (например, регуляризационный член). Это очень важное наблюдение — важно убедиться, что автоэнкодер не

просто учиться эффективным способом запоминать тренировочные данные. Мы хотим, чтобы он находил латентные признаки, чтобы быть полезным для данных, отличных от обучающего набора.

Существуют и другие способы создания узкого места в сети, кроме ограничения количества узлов.

2. Вероятностные основы VAE

В основе VAE лежит вероятностный подход:

- Цель: Найти распределение латентных переменных z , которое объясняет данные x .
- Гипотеза: Данные x генерируются из скрытых латентных переменных z , которые подчиняются некоторому вероятностному распределению $P(z)$.

2.1 Байесовская формулировка

Для решения задачи мы используем правило Байеса:

$$P(z|x) = \frac{P(x|z)P(z)}{P(x)}$$

Где:

- $P(z|x)$: апостериорное распределение латентной переменной z , определяющее, какие z вероятны для заданного x .
- $P(x|z)$: вероятность восстановления данных x из z (моделируется декодером).
- $P(z)$: априорное распределение z , обычно нормальное $N(0,1)$.
- $P(x)$: маргинальное распределение данных x .

2.2 Проблема вычислений

- Прямое вычисление $P(z|x)$ невозможно, так как интегрирование маргинального распределения $P(x)$ очень сложное:

$$P(x) = \int P(x|z)P(z)dz$$

2.3 Решение: Вариационная аппроксимация

Мы аппроксимируем сложное $P(z|x)$ через более простое распределение $Q(z|x)$, называемое вариационным распределением.

Цель — минимизировать разницу между $Q(z|x)$ и $P(z|x)$ с помощью KL-дивергенции:

$$KL(Q(z|x)||P(z|x)) = \int Q(z|x) \log \frac{Q(z|x)}{P(z|x)} dz$$

3. Архитектура VAE

3.1 Энкодер

Энкодер $Q(z|x)$:

- Принимает данные x и возвращает параметры распределения в латентном пространстве:
 - Среднее (μ).
 - Логарифм дисперсии ($\log \sigma^2$).

3.2 Латентное пространство

- Латентное пространство z моделируется как вероятностное распределение с параметрами μ и σ , полученными от энкодера.
- Генерация:

$$z = \mu + \epsilon \cdot \sigma,$$

где $\epsilon \sim \mathcal{N}(0, 1)$.

3.3 Декодер

Декодер $P(x|z)$:

- Преобразует z обратно в вероятностное представление данных x .
- Использует слои нейронной сети для моделирования распределения.

4. Функция потерь

Обучение VAE сводится к минимизации следующей функции:

$$\mathcal{L} = \mathbb{E}_{Q(z|x)} [\log P(x|z)] - KL(Q(z|x) || P(z))$$

4.1 Реконструкционная ошибка

Первая часть:

$$\mathbb{E}_{Q(z|x)} [\log P(x|z)]$$

- Отвечает за качество реконструкции данных.
- Меряет разницу между входом x и его реконструкцией x' .

4.2 KL-дивергенция

Вторая часть:

$$KL(Q(z|x) || P(z))$$

- Минимизирует расхождение между распределением латентных переменных $Q(z|x)$ и априорным распределением $P(z)$.

5. Интуиция и визуализация

1. Латентное пространство:

- Каждая точка z соответствует одному набору характеристик данных.
- Пример: для набора изображений MNIST разные точки могут соответствовать различным стилям написания цифр.

2. Интерполяция:

- Переход между двумя точками z_1 и z_2 в латентном пространстве позволяет плавно изменять характеристики данных.

3. Генерация новых данных:

- Выбор точки из $N(0,1)$ в латентном пространстве позволяет создать новые, не существовавшие ранее примеры.

6. Применение VAE

1. Компьютерное зрение:

- Генерация изображений.
- Удаление шума (denoising).

2. Обработка текста:

- Генерация текста.
- Обучение скрытых представлений слов.

3. Анализ данных:

- Снижение размерности.
- Визуализация данных.

4. Медицина:

- Генерация данных пациентов для аугментации в задачах диагностики.

5. Аудио:

- Генерация звуковых последовательностей.

7. Расширения VAE

1. Conditional VAE (cVAE):

- Добавление условной информации (например, класса данных) для генерации контролируемых примеров.

2. β -VAE:

- Вариация с дополнительным весом для KL-дивергенции, улучшает интерпретацию латентного пространства.

3. Hierarchical VAE:

- Использует многоуровневое латентное пространство для повышения точности.

4. Vector Quantized VAE (VQ-VAE):

- Использует дискретное латентное пространство для задач, требующих категориальной информации.

Цель

В данной лабораторной работе изучим принципы работы вариационных автокодировщиков (VAE) и их применение для генерации новых данных. Также будет проведен анализ латентного пространства и исследованы возможности его интерпретации.

Список задач лабораторной работы

1. Подготовка данных:

- Загрузить и предобработать датасет (например, MNIST).

- Нормализовать данные для их приведения к диапазону $[0, 1]$.
 - Разделить данные на обучающую и тестовую выборки.
 - Визуализировать данные и изучить их структуру.
2. Изучение архитектуры VAE:
 - Разобраться в компонентах VAE: энкодер, декодер, латентное пространство.
 - Понять особенности функции потерь VAE, включая реконструкционную ошибку и KL-дивергенцию.
 3. Реализация автокодировщика (AE):
 - Создать классический автокодировщик для MNIST.
 - Провести обучение и визуализировать результаты реконструкции.
 4. Реализация вариационного автокодировщика (VAE):
 - Реализовать архитектуру VAE.
 - Провести обучение и визуализировать латентное пространство.
 5. Сравнение PCA, AE и VAE:
 - Реализовать PCA для уменьшения размерности данных.
 - Сравнить результаты реконструкции данных между PCA, AE и VAE.
 - Оценить латентное пространство и вычислить метрики качества.
 6. Генерация данных:
 - Сгенерировать изображения из случайных точек латентного пространства VAE.
 - Провести интерполяцию между двумя точками латентного пространства.
 - Исследовать влияние размерности латентного пространства на качество генерации.
 - Добавить шум в латентное пространство и оценить влияние на генерацию.
 7. (Optional) Сравнение VAE и GAN:
 - Реализовать простую архитектуру GAN для генерации изображений.
 - Сравнить качество изображений, сгенерированных VAE и GAN.
 - Провести интерполяцию в латентном пространстве GAN.
 8. Анализ результатов:
 - Построить графики зависимости метрик качества от числа эпох для AE, VAE и GAN (Optional).
 - Сравнить латентное пространство и характеристики генерации данных.

Шаг 1: Подготовка данных

Теория: Набор данных MNIST состоит из изображений рукописных цифр (28x28 пикселей) в оттенках серого. Данные хранятся в виде массива чисел от 0 до 255. Чтобы улучшить работу нейронных сетей, важно нормализовать данные в диапазон $[0, 1]$. Это ускоряет обучение, делает градиенты более устойчивыми и предотвращает переполнения.

Код:


```

from tensorflow.keras.datasets import mnist
import numpy as np

# Загрузка данных
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Нормализация и преобразование данных
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))

print(f"Форма тренировочных данных: {x_train.shape}")
print(f"Форма тестовых данных: {x_test.shape}")

```

Задачи:

1. Проверка структуры данных.

- Задача: Убедитесь, что тренировочные и тестовые данные имеют правильные размеры.
- Код:

```

print(f"Размер тренировочного набора: {x_train.shape}, метки: {y_train.shape}")
print(f"Размер тестового набора: {x_test.shape}, метки: {y_test.shape}")

```

2. Визуализация данных.

- Задача: Постройте несколько изображений из тренировочного набора, чтобы визуально оценить данные.
- Код:

```

import matplotlib.pyplot as plt

```

Визуализация первых 10 изображений

```

plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(x_train[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()

```

3. Распределение классов.

- Задача: Подсчитайте, сколько примеров каждого класса (цифры 0–9) содержится в тренировочном и тестовом наборах.
- Код:

```

unique, counts = np.unique(y_train, return_counts=True)

```

```
print(f"Распределение классов в тренировочном наборе: {dict(zip(unique, counts))}")
unique, counts = np.unique(y_test, return_counts=True)
print(f"Распределение классов в тестовом наборе: {dict(zip(unique, counts))}")
```

4. Аугментация данных.

- Задача: Создайте новые тренировочные примеры, используя вращение, сдвиг и другие преобразования.
- Код:

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
# Настройки аугментации
```

```
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.1
)
datagen.fit(x_train)
```

```
# Визуализация аугментированных данных
```

```
augmented_images, _ = next(datagen.flow(x_train, y_train, batch_size=10))
plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(augmented_images[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()
```

5. Преобразование данных в одномерный массив.

- Задача: Преобразуйте изображения в одномерные массивы, чтобы подготовить данные для PCA.
- Код:

```
x_train_flat = x_train.reshape(len(x_train), -1)
x_test_flat = x_test.reshape(len(x_test), -1)
print(f"Размер после преобразования: {x_train_flat.shape}")
```

6. Статистический анализ пикселей.

- Задача: Вычислите минимальное, максимальное, среднее и стандартное отклонение пикселей изображений.
- Код:

```
print(f"Минимум: {x_train.min()}, Максимум: {x_train.max()}")
print(f"Среднее: {x_train.mean():.2f}, Стандартное отклонение: {x_train.std():.2f}")
```

7. Добавление шума в данные.

- Задача: Добавьте к изображениям гауссовский шум для экспериментов с моделями шумоподавления.
- Код:

```
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_test.shape)
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
```

Визуализация

```
plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(x_train_noisy[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()
```

8. Создание небольшого поднабора данных.

- Задача: Для быстрой проверки и тестирования моделей создайте небольшой поднабор тренировочных данных.
- Код:

```
small_x_train = x_train[:1000]
small_y_train = y_train[:1000]
print(f"Размер небольшого поднабора: {small_x_train.shape}")
```

Дополнительные вопросы для анализа:

1. Как распределение классов влияет на обучение модели?
2. Какие преобразования данных могут улучшить генерацию изображений в дальнейшем?
3. Как добавление шума может помочь в обучении устойчивых моделей?

Шаг 2: Реализация автокодировщика (АЕ)

Теория: Классический автокодировщик состоит из двух частей: кодировщика и декодировщика. Его цель — сжать данные в латентное пространство и восстановить их обратно.

Код:

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Reshape
```

```
# Архитектура AE
input_img = Input(shape=(28, 28, 1))
x = Flatten()(input_img)
encoded = Dense(64, activation='relu')(x)
decoded = Dense(28 * 28, activation='sigmoid')(encoded)
decoded = Reshape((28, 28, 1))(decoded)

# Создание модели
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# Обучение
autoencoder.fit(x_train, x_train, epochs=10, batch_size=256, validation_split=0.1)
```

Задача:

1. Обучите автокодировщик на наборе данных MNIST.
2. Визуализируйте реконструкцию изображений до и после обучения.

Шаг 3: Реализация вариационного автокодировщика (VAE)

Теория: Вариационный автокодировщик (VAE) добавляет вероятностную природу к латентному пространству.

Код:

```
from tensorflow.keras.layers import Lambda, Layer
from tensorflow.keras import backend as K

# Функция потерь VAE
def vae_loss(input_img, output):
    reconstruction_loss = K.sum(K.binary_crossentropy(K.flatten(input_img),
K.flatten(output)))
    kl_loss = -0.5 * K.sum(1 + log_var - K.square(mu) - K.exp(log_var))
    return reconstruction_loss + kl_loss

# Энкодер
latent_dim = 2
input_img = Input(shape=(28, 28, 1))
x = Flatten()(input_img)
x = Dense(128, activation='relu')(x)
mu = Dense(latent_dim)(x)
log_var = Dense(latent_dim)(x)

def sampling(args):
    mu, log_var = args
    epsilon = K.random_normal(shape=K.shape(mu))
    return mu + K.exp(0.5 * log_var) * epsilon

z = Lambda(sampling, output_shape=(latent_dim,))([mu, log_var])
```

```

# Декодер
decoder_input = Input(shape=(latent_dim,))
decoder_h = Dense(128, activation='relu')(decoder_input)
decoder_output = Dense(28 * 28, activation='sigmoid')(decoder_h)
decoder_output = Reshape((28, 28, 1))(decoder_output)
decoder = Model(decoder_input, decoder_output)

output = decoder(z)
vae = Model(input_img, output)
vae.add_loss(vae_loss(input_img, output))
vae.compile(optimizer='adam')

```

Обучение VAE

```
vae.fit(x_train, epochs=10, batch_size=256, validation_split=0.1)
```

Задача:

1. Реализуйте архитектуру VAE.
2. Визуализируйте латентное пространство, преобразовав входные данные.

Шаг 4: Сравнение PCA, AE и VAE

Теория

Метод главных компонент (PCA) — это алгоритм уменьшения размерности, который работает на основе линейной алгебры. PCA преобразует данные в новое пространство меньшей размерности, при этом максимизируя сохранение дисперсии исходных данных. В отличие от PCA, автокодировщики (AE) и вариационные автокодировщики (VAE) используют нелинейные преобразования для кодирования данных, что позволяет лучше работать с более сложными структурами.

Код

```

from sklearn.decomposition import PCA

# Преобразование данных в одномерный массив
x_train_flat = x_train.reshape(len(x_train), -1)
x_test_flat = x_test.reshape(len(x_test), -1)

# Реализация PCA
pca = PCA(n_components=64)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)

# Обратное преобразование для визуализации реконструкции

```

```
x_train_reconstructed = pca.inverse_transform(x_train_pca)
x_test_reconstructed = pca.inverse_transform(x_test_pca)
```

Задача 1: Визуализация и сравнение реконструкции

1. Визуализируйте оригинальные изображения и их реконструкции после PCA.
2. Сравните качество реконструкции с автокодировщиком (AE) и вариационным автокодировщиком (VAE).

Код:

```
import matplotlib.pyplot as plt

# Визуализация оригинальных и восстановленных изображений
def visualize_reconstruction(original, reconstructed, num_images=10):
    plt.figure(figsize=(10, 4))
    for i in range(num_images):
        # Оригинал
        plt.subplot(2, num_images, i + 1)
        plt.imshow(original[i].reshape(28, 28), cmap='gray')
        plt.axis('off')

        # Восстановление
        plt.subplot(2, num_images, num_images + i + 1)
        plt.imshow(reconstructed[i].reshape(28, 28), cmap='gray')
        plt.axis('off')
    plt.show()

# Визуализация для PCA
visualize_reconstruction(x_test_flat, x_test_reconstructed)
```

Вопросы для анализа:

- Какие отличия в реконструкции изображений наблюдаются между PCA, AE и VAE?
- Почему PCA не всегда хорошо справляется с восстановлением сложных структур?

Задача 2: Оценка метрик качества

1. Вычислите метрику качества реконструкции (например, среднеквадратичную ошибку, MSE) для PCA, AE и VAE.
2. Сравните метрики между этими методами.

Код:

```
from sklearn.metrics import mean_squared_error

# Оценка MSE для PCA
mse_pca = mean_squared_error(x_test_flat, x_test_reconstructed)
print(f"MSE для PCA: {mse_pca}")
```

```
# Оценка MSE для AE и VAE
# Для этого предварительно обучите модели AE и VAE
# mse_ae = mean_squared_error(x_test_flat, x_test_ae_reconstructed)
# mse_vae = mean_squared_error(x_test_flat, x_test_vae_reconstructed)
```

```
# Сравнение
# print(f"MSE для AE: {mse_ae}")
# print(f"MSE для VAE: {mse_vae}")
```

Вопросы для анализа:

- Почему метрика MSE у PCA может быть хуже, чем у AE и VAE?
- В каких случаях PCA может показать лучшие результаты?

Задача 3: Визуализация латентного пространства

1. Визуализируйте латентное пространство, полученное с помощью PCA, AE и VAE, для уменьшения размерности до 2D.
2. Используйте классы цифр как метки для цветовой кодировки.

Код:

```
# Уменьшение размерности до 2D с PCA
pca_2d = PCA(n_components=2)
x_train_pca_2d = pca_2d.fit_transform(x_train_flat)

# Визуализация латентного пространства
plt.figure(figsize=(8, 6))
plt.scatter(x_train_pca_2d[:, 0], x_train_pca_2d[:, 1], c=y_train, cmap='tab10', s=1)
plt.colorbar()
plt.title("Латентное пространство PCA (2D)")
plt.show()
```

Вопросы для анализа:

- Как выглядит латентное пространство PCA по сравнению с AE и VAE?
- Почему линейное пространство PCA не всегда оптимально для сложных данных?

Задача 4: Оценка сохранения дисперсии

1. Вычислите долю сохраненной дисперсии для разных значений `n_components` в PCA.
2. Постройте график зависимости доли дисперсии от числа компонент.

Код:

```
# Оценка сохраненной дисперсии
explained_variance = []
components_range = range(1, 65)
for n in components_range:
    pca = PCA(n_components=n)
```

```
pca.fit(x_train_flat)
explained_variance.append(sum(pca.explained_variance_ratio_))
```

```
# Построение графика
plt.figure(figsize=(8, 6))
plt.plot(components_range, explained_variance, marker='o')
plt.title("Зависимость сохраненной дисперсии от числа компонент")
plt.xlabel("Число компонент")
plt.ylabel("Доля сохраненной дисперсии")
plt.grid()
plt.show()
```

Вопросы для анализа:

- Как выбрать оптимальное количество компонент для PCA?
- Почему сохранение дисперсии не всегда означает хорошее качество реконструкции?

Задача 5: Применение PCA в комбинации с другим методом

1. Используйте PCA как предварительный этап перед применением АЕ или VAE.
2. Оцените, улучшилось ли качество реконструкции при использовании комбинации методов.

Код:

```
# Сжатие данных с помощью PCA перед обучением АЕ
x_train_pca_ae = pca.fit_transform(x_train_flat)
x_train_pca_ae = x_train_pca_ae.reshape(len(x_train_pca_ae), -1)
```

```
# Обучение АЕ на данных, обработанных PCA (код АЕ реализуйте ранее)
autoencoder.fit(x_train_pca_ae, x_train_pca_ae, epochs=10, batch_size=256)
```

Вопросы для анализа:

- В каких случаях использование PCA как предварительного этапа может быть полезным?
- Как комбинирование методов может улучшить результаты?

Шаг 5: Генерация данных

Теория

Генерация данных — ключевая задача для вариационных автокодировщиков (VAE). Благодаря вероятностной природе латентного пространства VAE могут генерировать новые данные, которые напоминают те, что были использованы для обучения. Латентное пространство позволяет интерполировать между различными представлениями, создавая плавные переходы между данными.

Код


```

import numpy as np
import matplotlib.pyplot as plt

# Генерация случайных точек в латентном пространстве
latent_samples = np.random.normal(size=(10, latent_dim))

# Генерация изображений с помощью декодера
generated_images = decoder.predict(latent_samples)

# Визуализация сгенерированных изображений
plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(generated_images[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()

```

Подробные задачи

Задача 1: Сравнение сгенерированных изображений

1. Генерируйте изображения из случайных точек латентного пространства и сравните их с оригинальными.
2. Оцените, насколько хорошо сгенерированные изображения напоминают данные MNIST.

Код:

```

# Сравнение оригиналов и генерации
plt.figure(figsize=(10, 4))
for i in range(10):
    # Оригинальные данные
    plt.subplot(2, 10, i + 1)
    plt.imshow(x_train[i].reshape(28, 28), cmap='gray')
    plt.axis('off')

    # Сгенерированные данные
    plt.subplot(2, 10, 10 + i + 1)
    plt.imshow(generated_images[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()

```

Вопросы для анализа:

- Насколько сгенерированные изображения напоминают цифры?
- Какие недостатки наблюдаются в генерации?

Задача 2: Интерполяция в латентном пространстве

1. Выберите две точки в латентном пространстве.

2. Проведите интерполяцию между этими точками и визуализируйте промежуточные изображения.

Код:

```
# Выбор двух точек в латентном пространстве
point_a = np.random.normal(size=(1, latent_dim))
point_b = np.random.normal(size=(1, latent_dim))

# Генерация промежуточных точек
interpolation_steps = 10
interpolated_points = np.linspace(point_a, point_b, interpolation_steps)

# Генерация изображений
interpolated_images = decoder.predict(interpolated_points)

# Визуализация
plt.figure(figsize=(10, 2))
for i in range(interpolation_steps):
    plt.subplot(1, interpolation_steps, i + 1)
    plt.imshow(interpolated_images[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()
```

Вопросы для анализа:

- Какие изменения в изображениях наблюдаются в процессе интерполяции?
- Как плавность интерполяции зависит от структуры латентного пространства?

Задача 3: Исследование структуры латентного пространства

1. Сгенерируйте сетку точек в 2D-латентном пространстве.
2. Визуализируйте сгенерированные изображения для каждой точки сетки.

Код:

```
# Генерация сетки в латентном пространстве
grid_x = np.linspace(-3, 3, 10)
grid_y = np.linspace(-3, 3, 10)
latent_grid = np.array([[x, y] for x in grid_x for y in grid_y])

# Генерация изображений
grid_images = decoder.predict(latent_grid)

# Визуализация
plt.figure(figsize=(10, 10))
for i, img in enumerate(grid_images):
    plt.subplot(10, 10, i + 1)
    plt.imshow(img.reshape(28, 28), cmap='gray')
```

```
plt.axis('off')
plt.show()
```

Вопросы для анализа:

- Как изображения изменяются при перемещении по сетке латентного пространства?
- Какие области латентного пространства порождают несоответствующие данные?

Задача 4: Добавление шума

1. Добавьте случайный шум к точкам латентного пространства и оцените, как это влияет на генерацию изображений.
2. Проведите эксперимент с разными уровнями шума.

Код:

```
# Добавление шума
noise_level = 0.5
noisy_points = latent_samples + np.random.normal(scale=noise_level,
size=latent_samples.shape)
```

```
# Генерация изображений
noisy_images = decoder.predict(noisy_points)
```

```
# Визуализация
plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(noisy_images[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()
```

Вопросы для анализа:

- Как шум влияет на качество сгенерированных изображений?
- Какие уровни шума допустимы, чтобы сохранить четкость изображения?

Задача 5: Влияние размерности латентного пространства

1. Проведите эксперименты с разными размерностями латентного пространства (например, 2, 8, 16).
2. Сравните качество сгенерированных изображений.

Код:

```
# Перезапуск модели с разной размерностью латентного пространства
latent_dim_options = [2, 8, 16]
for latent_dim in latent_dim_options:
    print(f"Размер латентного пространства: {latent_dim}")
    # Обучение новой модели с измененной размерностью
    # ...
```

```

# Генерация изображений
latent_samples = np.random.normal(size=(10, latent_dim))
generated_images = decoder.predict(latent_samples)

# Визуализация
plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(generated_images[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()

```

Вопросы для анализа:

- Как размерность латентного пространства влияет на качество генерации?
- Чем вызвано ухудшение или улучшение результатов при изменении размерности?

(Optional) Задача 6: Сравнение с GAN

1. Реализуйте простую архитектуру GAN для генерации изображений.
2. Сравните результаты генерации GAN и VAE.

Код: Реализация GAN

Генеративно-состязательная сеть (GAN) — это архитектура, состоящая из двух частей: генератора, создающего данные, и дискриминатора, отличающего реальные данные от сгенерированных.

```

from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense, Flatten, Reshape, LeakyReLU,
BatchNormalization, Input
from tensorflow.keras.optimizers import Adam
import numpy as np
import matplotlib.pyplot as plt

```

```

# Параметры
latent_dim = 100

```

```

# Генератор
def build_generator(latent_dim):
    model = Sequential([
        Dense(128, activation='relu', input_dim=latent_dim),
        BatchNormalization(),
        Dense(256, activation='relu'),
        BatchNormalization(),
        Dense(28 * 28, activation='sigmoid'),
        Reshape((28, 28, 1))
    ])
    return model

```

```

# Дискриминатор

```

```
def build_discriminator():
    model = Sequential([
        Flatten(input_shape=(28, 28, 1)),
        Dense(256),
        LeakyReLU(alpha=0.2),
        Dense(128),
        LeakyReLU(alpha=0.2),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer=Adam(learning_rate=0.0002), loss='binary_crossentropy',
metrics=['accuracy'])
    return model
```

Построение GAN

```
generator = build_generator(latent_dim)
discriminator = build_discriminator()
discriminator.trainable = False
```

```
gan_input = Input(shape=(latent_dim,))
gan_output = discriminator(generator(gan_input))
gan = Model(gan_input, gan_output)
gan.compile(optimizer=Adam(learning_rate=0.0002), loss='binary_crossentropy')
```

Обучение GAN

```
def train_gan(generator, discriminator, gan, epochs, batch_size, latent_dim):
    for epoch in range(epochs):
        # Генерация случайного шума
        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        generated_images = generator.predict(noise)

        # Подготовка реальных данных
        idx = np.random.randint(0, x_train.shape[0], batch_size)
        real_images = x_train[idx]

        # Создание меток
        real_labels = np.ones((batch_size, 1))
        fake_labels = np.zeros((batch_size, 1))

        # Обучение дискриминатора
        d_loss_real = discriminator.train_on_batch(real_images, real_labels)
        d_loss_fake = discriminator.train_on_batch(generated_images, fake_labels)

        # Обучение генератора
        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        g_loss = gan.train_on_batch(noise, real_labels)

        # Вывод метрик каждые 100 эпох
```

```

if epoch % 100 == 0:
    print(f"Эпоха {epoch}: [D loss: {d_loss_real[0]}] [G loss: {g_loss}]")

# Запуск обучения
train_gan(generator, discriminator, gan, epochs=5000, batch_size=64,
latent_dim=latent_dim)

# Генерация новых изображений
noise = np.random.normal(0, 1, (10, latent_dim))
generated_images = generator.predict(noise)

# Визуализация
plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(generated_images[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()

```

Задача: Сравнение VAE и GAN

1. Реализуйте и обучите GAN для генерации изображений MNIST.
2. Сравните качество изображений, сгенерированных VAE и GAN.
3. Проведите интерполяцию в латентном пространстве GAN.